

THE SERVICE LAYER

Práctica 10 – Entorno Servidor

ANDRÉS ALBARRACÍN, DAVID

Tabla de contenido

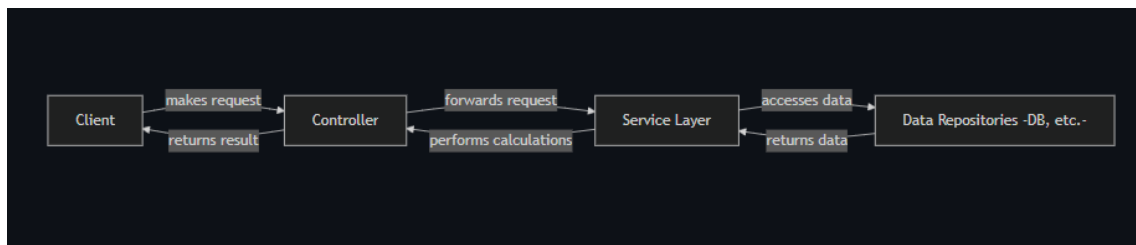
The Service Layer	2
1. Funciones de la capa de servicio	2
2. Implementación de una clase de servicio	2
3. Acoplamiento y cómo resolverlo	4
Actividad	5

The Service Layer

1. Funciones de la capa de servicio

La capa de servicio es responsable de la lógica de negocio, las reglas y los cálculos que la aplicación debe realizar en cada caso.

Normalmente, los métodos de una clase de servicio se invocan desde un controlador, que es el encargado de pasar los valores proporcionados por el cliente al servicio como parámetros para realizar los cálculos necesarios. También es común que el servicio necesite acceder a repositorios de datos para obtener información adicional requerida para ejecutar las reglas de negocio. Finalmente, los métodos de la capa de servicio devuelven el resultado al controlador, que a su vez lo envía de vuelta al cliente.



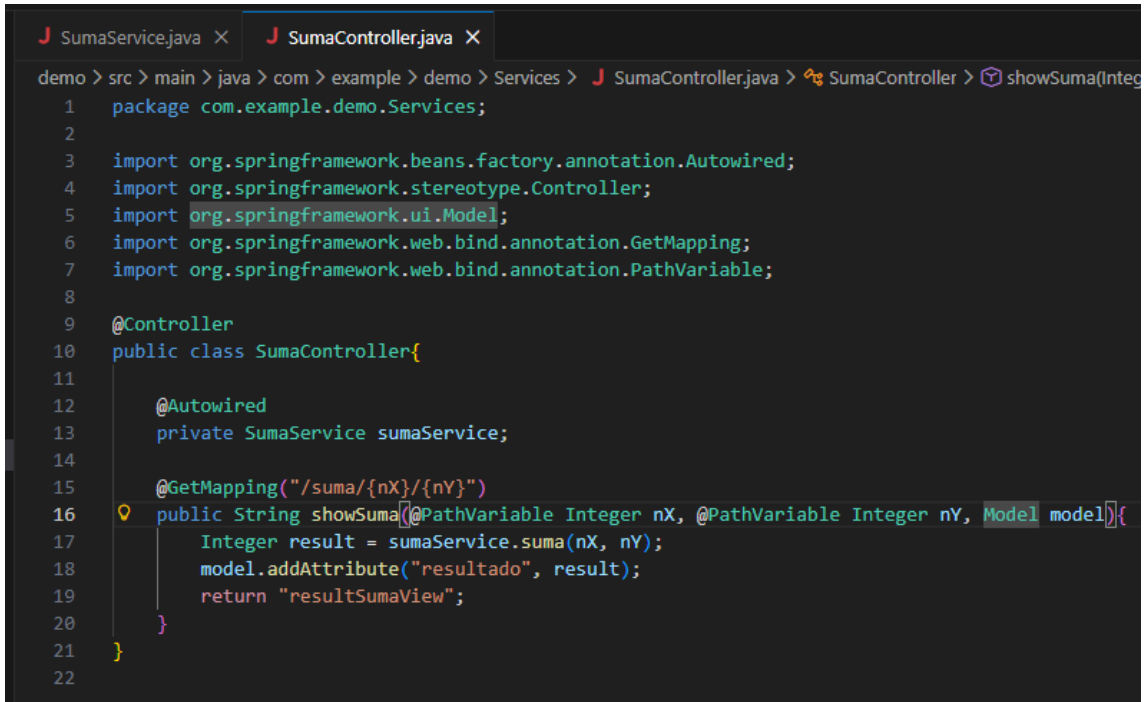
2. Implementación de una clase de servicio

Las clases de servicio se anotan con **@Service**, y en lugar de crear instancias en el controlador con un constructor como hemos hecho hasta ahora, Spring inyecta el servicio en el controlador utilizando la **@Autowired** anotación, reduciendo así el **acoplamiento del código**.

En cuanto a la ubicación de los servicios, podemos crear un paquete `services` y almacenarlos allí, o bien, si hemos optado por una organización más orientada al dominio, pueden ir en la misma carpeta que los controladores de la misma área (**Cliente**, **Producto**, etc.). Spring no impone restricciones a esta estructura, pero deben ser subpaquetes del paquete raíz, al igual que los controladores.

```

SumaService.java X SumaController.java
demo > src > main > java > com > example > demo > Services > SumaService.java > SumaService
1  package com.example.demo.Services;
2
3  import org.springframework.stereotype.Service;
4
5  @Service
6  public class SumaService {
7      public Integer suma(Integer a, Integer b){
8          return a + b;
9      }
10 }
11
  
```



```
1 package com.example.demo.Services;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.stereotype.Controller;
5 import org.springframework.ui.Model;
6 import org.springframework.web.bind.annotation.GetMapping;
7 import org.springframework.web.bind.annotation.PathVariable;
8
9 @Controller
10 public class SumaController{
11
12     @Autowired
13     private SumaService sumaService;
14
15     @GetMapping("/suma/{nX}/{nY}")
16     public String showSuma(@PathVariable Integer nX, @PathVariable Integer nY, Model model){
17         Integer result = sumaService.suma(nX, nY);
18         model.addAttribute("resultado", result);
19         return "resultSumaView";
20     }
21 }
22
```

Podemos observar que al agregar **@Autowired**, ya no estamos llamando manualmente al constructor del servicio.

Los objetivos de la capa de servicio son:

- Para separar la lógica de negocio de la presentación de datos. Cualquier cambio en la forma en que se reciben o muestran los datos (por ejemplo, cambiar a una API REST) no afectará a esta parte de la aplicación.
- Código más estructurado, con métodos y clases más pequeños.
- Pruebas unitarias más sencillas: podemos probar diferentes servicios sin necesidad de simular todo el proceso de interacción del usuario.

Además, el uso de servicios a través de Spring presenta varias ventajas con respecto a la gestión manual de datos, como hemos hecho hasta ahora. Spring se encarga del acceso, la instanciación y la destrucción de los servicios, lo que ayuda a prevenir errores relacionados con el acoplamiento.

3. Acoplamiento y cómo resolverlo

El acoplamiento es la dependencia entre clases y puede resolverse utilizando un enfoque basado en componentes.

```
class Motor{
    int id;
    int potencia;
    Motor(int id, int potencia){
        this.id = id;
        this.potencia = potencia;
    }
}

// CAR CLASS WITH COUPLING
class Coche{
    String modelo;
    Motor motor;
    Coche (String modelo, int i, int p){
        this.modelo = modelo;
        this.motor = new Motor(i, p);
    }
}

// CAR CLASS WITH DEPENDENCY INJECTION (LESS COUPLING)
class Coche{
    String modelo;
    Motor motor;
    Coche (String modelo, Motor motor){
        this.modelo = modelo;
        this.motor = motor;
    }
}
```

Spring implementa la **Inversión de Control (IoC)** y la **Inyección de Dependencias (DI)** para reducir el acoplamiento. La clase no crea sus dependencias; Spring las proporciona.

Actividad

- Modificar la clase de gestión de datos para convertirla en un servicio.

```
@Service
public class SCVServices {

    /**
     * Creamos una línea nueva en el CSV con el compositor
     * @param composerAsCSV Nuevo compositor en String
     * @param csvPath Ruta del archivo donde se guarda el compositor
     */
    public void create(String composerAsCSV, String csvPath) {
        try (PrintWriter printWriter = new PrintWriter(new File(csvPath))) {
            printWriter.println(composerAsCSV);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    /**
     * Buscar la línea del compositor y no escribirla de nuevo, esto sobrescribe el CSV
     * @param composerAsCSV El compositor que queremos borrar
     * @param csvPath Ruta del archivo donde se guardan los compositores menos el que vamos a borrar
     */
    public void delete(String composerAsCSV, String csvPath) {
        try (BufferedReader reader = new BufferedReader(new FileReader(new File(csvPath)))) {
            while (!reader.readLine().matches(composerAsCSV)) {
                create(composerAsCSV, csvPath);
            }
            reader.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

```

/**
 * Creamos un string para guardar la línea de compositores y la mostramos en consola.
 * @param csvPath Ruta del archivo que guarda los compositores
 */
public void read(String csvPath) {
    String linea = "";
    try (BufferedReader reader = new BufferedReader(new FileReader(new File(csvPath)))) {
        while ((linea = reader.readLine()) != null) {
            System.out.println(linea);
        }
        reader.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}

/**
 * Recibir un compositor viejo, el nuevo con las modificaciones y la ruta del archivo
 * @param composerAsCSV El nuevo compositor
 * @param oldComposer El compositor que queremos eliminar por las modificaciones
 * @param csvPath La ruta donde están los compositores
 */
public void update(String composerAsCSV, String oldComposer, String csvPath) {
    try (BufferedReader reader = new BufferedReader(new FileReader(new File(csvPath)))) {
        while (reader.readLine().matches(oldComposer)) {
            delete(oldComposer, csvPath);
            create(composerAsCSV, csvPath);
        }
        reader.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}

```